Generalization-Focused Deep Reinforcement Learning for Traffic Signal Control: Comparing DQN, Double DQN, PPO and A2C on Real-City Scenarios

Lakshitha Konara
Department of Computer Science and Engineering
University of Moratuwa
Moratuwa, Sri Lanka
lakshithakonara3@gmail.com

Dulani Meedeniya
Department of Computer Science and Engineering
University of Moratuwa
Moratuwa, Sri Lanka
dulanim@cse.mrt.ac.lk

***Abstract*—Deep reinforcement learning (RL) controllers for adaptive traffic signals are often trained and tested on the same hour of traffic, which hides whether they generalize beyond it. This paper presents a generalization-focused training and evaluation framework for RL signal control on real-city scenarios and uses it to compare four deep RL algorithms: DQN, Double DQN, PPO and A2C. The framework trains on seeded demand variants of the real peak hour (0.8–$1.2\times$), selects models on separate validation variants, and controls heterogeneous intersections with a single shared policy. We use three RESCO benchmark scenarios from Cologne and Ingolstadt simulated in SUMO. All learners are compared with fixed-time and tuned actuated control over five training seeds, on test demand never seen in training ($0.7\times$ and $1.3\times$) and on the original real hour. The best learner, [TBD], reduces the average total waiting time per vehicle by [TBD] % relative to fixed-time control on unseen demand. An ablation study quantifies the contribution of demand-randomized training and of policy sharing.**

***Index Terms*—traffic signal control, deep reinforcement learning, generalization, DQN, PPO, SUMO**

I. Introduction

Traffic congestion at signalized intersections costs travel time, fuel and safety in cities worldwide. Most intersections still run fixed-time plans, which repeat the same cycle regardless of traffic, or actuated control, which stretches or cuts a green phase when loop detectors report arriving vehicles [1], [2]. Deep reinforcement learning (RL) offers a data-driven alternative. An agent at each intersection observes the queues on its approaches and chooses the next green phase, learning a policy that reduces delay [3]–[6]. On realistic city scenarios, decentralized deep Q-learning has proved a strong and robust baseline [7].

Two gaps make it hard to tell how such controllers would perform in practice. First, *evaluation rarely separates training and test demand*. A scenario typically ships one hour of calibrated traffic, and many studies train and report on that same hour, so a controller can perform well by memorizing one demand pattern [7]–[9]. Second, *algorithm comparisons are rarely controlled*. Learners are reported with different observations, rewards, yellow times, tuning effort and numbers of seeds, so differences between them can come from the setup rather than from the algorithm [10]. It is therefore unclear which of the widely used deep RL algorithms generalizes best to traffic it has not seen.

This paper addresses the first stage of a larger study on resource-aware edge–cloud intelligent transportation systems: identifying and training an RL signal controller that generalizes, benchmarked against fixed-time and actuated control. The contributions are:

- A **generalization-focused training framework** with two components: demand-randomized training with validation-based model selection (C1), and a single policy shared across heterogeneous intersections (C2) (Section IV).
- A **controlled comparison of four deep RL algorithms** (DQN, Double DQN, PPO, A2C) against fixed-time, tuned actuated and random control on three RESCO scenarios. The comparison uses identical observations, rewards, yellow times and tuning budgets, and five training seeds per configuration.
- A **generalization protocol** with strictly separated train, validation and test demand. Training uses 0.8–$1.2\times$ variants of the real peak hour; testing uses unseen $0.7\times$ and $1.3\times$ demand and the original hour.
- An **ablation study** isolating the contribution of C1 and C2.

The rest of the paper is organized as follows. Section II reviews related work. Section III formulates the problem and Section IV presents the data pipeline and the framework. Section V describes the experimental setup, Section VI reports and discusses the results, and Section VII concludes.

II. Related Work

A. Deep RL for Traffic Signal Control

Early deep RL controllers learned phase selection from queue and image features at a single intersection [3]. Later

TABLE I
COMPARISON WITH RELATED STUDIES

Study	Approach	Unseen demand	Algorithm comparison
RESCO [7] (2021)	IDQN, IPPO, MPLight, FMA2C	×	✓
GeneraLight [8] (2020)	Meta-RL	✓	×
π-Light [19] (2024)	Programmatic policy	✓	×
Kwesiga *et al.* [9] (2026)	RL, ring-barrier	✓	×
Sim2Signal [20] (2026)	Sim-to-real benchmark	✓	✓
This work	DQN, DDQN, PPO, A2C	✓	✓

[TODO: verify every tick against the full papers before submission]

work introduced pressure-based states and rewards grounded in max-pressure control [4], [11], parameter sharing for thousands of intersections [5], and cooperative multi-agent actor-critic methods [6]; Wei *et al.* [2] survey the field. The RESCO benchmark [7] standardized evaluation on real-city SUMO [12] scenarios derived from TAPASCologne [13] and InTAS [14]. On those scenarios, independent DQN and PPO learners outperformed more specialized coordinated methods. Our study builds on this finding and compares value-based (DQN [15], Double DQN [16]) and policy-gradient (PPO [17], A2C [18]) learners under one protocol.

B. Generalization to Unseen Traffic

Because benchmarks usually provide one demand pattern, generalization is often untested. GeneraLight [8] used meta-RL across generated flow patterns to improve performance in unseen environments. π-Light [19] learns compact programmatic policies and reports better transfer across intersections in different cities than deep RL baselines. Kwesiga *et al.* [9] showed that a controller trained on a single origin–destination pattern generalizes only to similar demand, while training on diverse patterns improves robustness. Sim2Signal [20] benchmarks the simulation-to-reality gap of RL signal controllers along observation, action, transition and reward dimensions.

C. Research Gap

Table I positions this work. These studies either evaluate on unseen demand with a single learner or compare learners on the training demand. To our knowledge, no prior study compares several deep RL algorithm families under one controlled protocol on real-city scenarios, tests every learner on demand levels unseen in training, and isolates the contribution of each training component with an ablation.

III. PROBLEM FORMULATION

A road network contains a set $\mathcal{I}$ of signalized intersections. Each intersection $i \in \mathcal{I}$ is controlled by an agent that acts every $\Delta = 5\,\text{s}$, giving $T = 720$ decisions in a one-hour episode.

State. The observation of agent i at decision step t is

$$s_t^i = \big[\, \mathbf{e}(p_t^i),\ g_t^i,\ \rho_{1,t}^i, \ldots, \rho_{n_i,t}^i,\ q_{1,t}^i, \ldots, q_{n_i,t}^i \,\big], \qquad (1)$$

where $\mathbf{e}(p_t^i)$ is a one-hot encoding of the current green phase, $g_t^i \in \{0,1\}$ indicates whether the minimum green time has elapsed, and $\rho_{l,t}^i$ and $q_{l,t}^i$ are the vehicle density and the halting-vehicle density of incoming lane $l \in \{1, \ldots, n_i\}$, each normalized by the lane's capacity. For cologne1 (4 phases, 8 lanes) $s_t^i \in [0,1]^{21}$.

TABLE II
SCENARIOS (VERIFIED FROM THE NETWORK AND ROUTE FILES)

Scenario	Junct.	Roads / lanes / length	Signals (agents)	Green phases	Inputs per agent	Real-hour vehicles
cologne1	9	10 / 19 / 1.4 km	1	4	21	2,015
ingolstadt1	8	11 / 33 / 0.8 km	1	3	18	1,716
cologne3	29	48 / 76 / 5.7 km	3	3, 4, 4	14, 17, 21	2,856

Action. The agent selects the next green phase $a_t^i \in \{1, \ldots, P_i\}$. A phase change inserts a yellow interval of the same length as the city's signal plan and is permitted only after a minimum green of 5 s.

Reward. Let W_t^i be the accumulated waiting time of all vehicles on the incoming lanes of intersection i. The reward is the reduction in waiting time since the previous decision,

$$r_t^i = W_{t-1}^i - W_t^i, \qquad (2)$$

which is positive when queues are cleared and negative when they grow.

Objective. Each agent learns to maximize its expected discounted return $\mathbb{E}\big[\sum_t \gamma^t r_t^i\big]$ on training demand. The quantity of interest, however, is the average total waiting time per vehicle $\bar{w}$ on a test demand distribution $\mathcal{D}_{\text{test}}$ that differs from the training distribution $\mathcal{D}_{\text{train}}$:

$$\min_{\pi}\ \mathbb{E}_{D \sim \mathcal{D}_{\text{test}}}\big[\bar{w}(\pi, D)\big], \qquad \mathcal{D}_{\text{test}} \neq \mathcal{D}_{\text{train}}. \qquad (3)$$

Model selection may use only validation demand drawn from $\mathcal{D}_{\text{train}}$, never $\mathcal{D}_{\text{test}}$.

IV. METHODOLOGY

Fig. 1 summarizes the pipeline from input data to reported outputs.

A. Scenarios and Data

We use three real-city scenarios of the RESCO benchmark [7], distributed with the `sumo-rl` package [21] (Table II). The Cologne scenarios come from TAPASCologne [13] (morning peak, 07:00–08:00) and the Ingolstadt scenario from InTAS [14] (afternoon peak, 16:00–17:00). Each scenario provides a road network (`.net.xml`: junctions, lanes and the static signal programs) and one hour of trips (`.rou.xml`: departure time, origin and destination of every vehicle). Junctions without signals are dead ends, priority junctions or right-before-left junctions; only signalized junctions are agents.

B. Pre-processing: Demand Variants and Splits

Each scenario has only one hour of real demand, so training and testing on it would reward memorization. We therefore generate seeded variants of the real hour while keeping the road network unchanged. For a demand scale α, each trip is copied $n = \lfloor \alpha \rfloor + B$ times, where $B \sim \text{Bernoulli}(\alpha - \lfloor \alpha \rfloor)$.

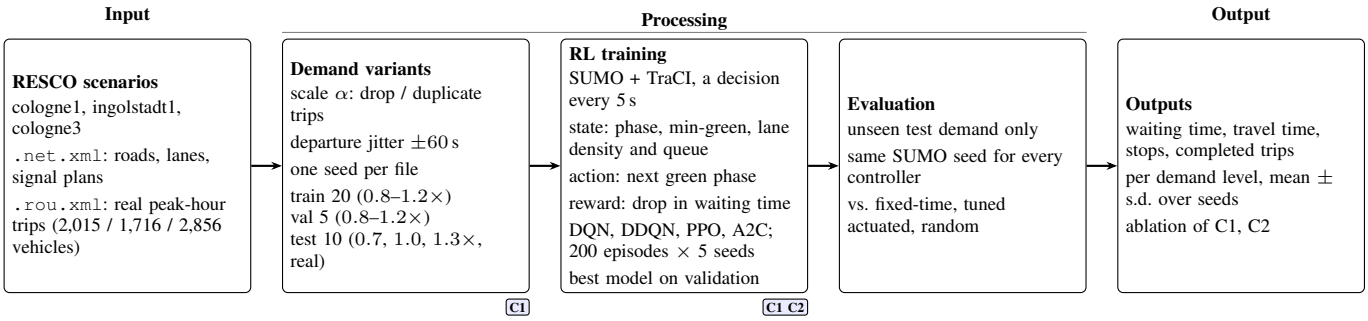


Fig. 1. End-to-end pipeline. Real peak-hour scenarios are expanded into seeded demand variants (train, validation and test kept separate). RL agents are trained in SUMO, selected on validation demand, and evaluated only on unseen test demand. C1 and C2 mark the framework components of Section IV-D.

TABLE III
TRAIN / VALIDATION / TEST SPLITS (ONE RECORD = ONE VEHICLE TRIP)

Scenario	Split	Files	Vehicles per file	Total records
cologne1	Train	20	1,599–2,434	40,290
	Val	5	1,613–2,422	10,045
	Test	10	1,390–2,649	20,156
ingolstadt1	Train	20	1,357–2,076	34,293
	Val	5	1,378–2,066	8,567
	Test	10	1,192–2,268	17,182
cologne3	Train	20	2,261–3,458	57,233
	Val	5	2,291–3,431	14,258
	Test	10	1,970–3,754	28,569

Trips are thus randomly dropped for $\alpha < 1$ and duplicated for $\alpha > 1$. Every departure time is then shifted by $U(-60, 60)$ s within the hour. Trips are sorted by departure, as SUMO requires, and an unused `arrival` attribute is removed. Each file uses its own random seed, so the variants differ but are exactly reproducible. Table III gives the resulting splits: training uses $\alpha \in \{0.8, 0.9, 1.0, 1.1, 1.2\}$ (4 variants each), validation the same levels (1 each), and test the unseen levels 0.7 and 1.3 plus 1.0 (3 variants each) and the unmodified real hour. One file is one episode; one record is one vehicle trip.

C. RL Learners

All learners use the state, action and reward of Section III and a multilayer perceptron with two hidden layers of 64 ReLU units. **DQN** [15] learns action values with experience replay and a target network. **Double DQN** [16] reduces DQN's over-estimation by selecting the next action with the online network and evaluating it with the target network:

$$y_t = r_t + \gamma\, Q_{\bar{\theta}}\big(s_{t+1}, \arg\max_a Q_\theta(s_{t+1}, a)\big). \qquad (4)$$

PPO [17] is an on-policy actor-critic method with a clipped surrogate objective. **A2C** [18] is the synchronous advantage actor-critic. Implementations follow Stable-Baselines3 [22].

D. Generalization-Focused Training Framework

The framework adds two components to a standard deep RL learner. Each can be removed independently for the ablation study.

C1 – Demand-randomized training with validation selection. Every episode loads the next training variant (each file once per shuffled pass), so the agent sees 0.8–1.2× demand. Every 10 episodes the greedy policy is run on the five validation files, and the checkpoint with the lowest validation waiting time is kept. Test files are never used for training or selection.

C2 – Shared policy for heterogeneous intersections. In multi-signal scenarios one network controls every signal. Inputs are zero-padded to the largest intersection and a one-hot intersection identifier is appended. Outputs use the largest phase count and are mapped to $a \bmod P_i$ for an intersection with P_i phases. Every signal therefore learns from the experience of all signals, and one network covers all intersection types. The alternative, one independent network per signal, is the ablation of C2.

[TODO: C2 ablation (independent policy per signal on cologne3) is not implemented yet; C1 ablation is (train on the real hour only).]

E. Baselines

Fixed-time control runs the static signal programs shipped with each scenario. **Actuated** control uses SUMO's gap-based actuation on the same phases. A green is extended while detectors see vehicles arriving within a maximum gap g, between a minimum green of 5 s and a maximum of κ times the planned green. For a fair comparison, (g, κ) is tuned on the validation files over $g \in \{2, 3, 4\}$ s and $\kappa \in \{1.5, 2, 3\}$. A **random** policy, choosing a random phase every 5 s, serves as a sanity check.

V. Experimental Setup

A. Simulation and Training

Table IV lists the settings shared by all controllers. The yellow interval equals that of each scenario's own signal plan, so RL gains no advantage from a shorter amber. Vehicles are never teleported, and vehicles unable to enter a congested network wait at the entry, so jams remain visible in the metrics. Each RL configuration is trained for 200 episodes (one simulated hour each) with five independent seeds and validated every 10 episodes. The validation-best checkpoint is evaluated once on the test files. All algorithms receive the

TABLE IV
SIMULATION SETTINGS AND HYPER-PARAMETERS

Simulation (all controllers)	
Simulator	SUMO 1.27.1 (libsumo), sumo-rl 1.4.5
Episode	3600 s peak hour, 720 decisions
Decision interval / min. green	5 s / 5 s
Yellow	as signal plan: 5 s cologne1, 3 s others
Teleporting / max. insertion delay	disabled / unlimited
RL (Stable-Baselines3 2.9.0, PyTorch)	
Network	MLP 64×64, ReLU; discount $\gamma = 0.99$
DQN, DDQN	lr 10^{-3}, buffer 50k, batch 64, target 500 steps, ϵ: $1\rightarrow0.02$ over first 10% of training
PPO	lr 3×10^{-4}, rollout 720, batch 144, 10 epochs, clip 0.2, GAE $\lambda = 0.95$, entropy 0.01
A2C	lr 7×10^{-4}, rollout 5, entropy 0.01
Budget	200 episodes $\times$ 5 seeds, validation every 10

same hyper-parameter tuning budget, using validation data only. Training ran on a workstation with **[TBD]** and **[TBD]** of RAM, with one run per CPU core.

B. Evaluation Metrics

Traffic metrics are computed from SUMO's per-vehicle trip records. Vehicles still driving or still waiting to enter at the end of the hour are *included*, so a controller cannot score well by blocking an entrance.

- **Average total waiting time** $\bar{w}$ (main metric): time halted on the road (speed below 0.1 m/s) plus time spent waiting to enter the network, averaged over vehicles.
- **Average travel time** and **average number of stops** per vehicle.
- **Completion rate**: percentage of vehicles that finish their trip, which reveals gridlock.
- **Generalization**: the reduction in $\bar{w}$ relative to fixed-time control at each test demand level. A controller generalizes if this reduction holds at the unseen $0.7\times$ and $1.3\times$ levels as well as at $1.0\times$.

Statistics. For each seed, metrics are averaged over the test files of a demand level. We report the mean $\pm$ standard deviation over five seeds [10], [23]. Differences are tested with Welch's t-test at $\alpha = 0.05$. All controllers use the same SUMO seed on a given test file. However, we observed that SUMO is not perfectly repeatable on cologne1 even with a fixed seed: identical fixed-time runs gave $\bar{w}$ of 30.0 s or 31.3 s. Single-episode differences of a few percent are therefore not interpreted.

VI. RESULTS AND DISCUSSION

A. Comparison of RL Algorithms

Table V compares all controllers over the ten test files of each scenario. [TODO: After evaluate.py: which algorithm is best per scenario; whether the differences are significant across seeds; how value-based and policy-gradient learners differ; whether actuated beats fixed-time (preliminary validation runs suggest it does on ingolstadt1 and cologne3 but not on cologne1).]

TABLE V
AVERAGE TOTAL WAITING TIME PER VEHICLE (S) ON ALL TEST FILES, MEAN $\pm$ S.D. OVER 5 SEEDS (LOWER IS BETTER)

Controller	cologne1	ingolstadt1	cologne3
Fixed-time	**[TBD]**	**[TBD]**	**[TBD]**
Actuated (tuned)	**[TBD]**	**[TBD]**	**[TBD]**
Random	**[TBD]**	**[TBD]**	**[TBD]**
DQN	**[TBD]**	**[TBD]**	**[TBD]**
Double DQN	**[TBD]**	**[TBD]**	**[TBD]**
PPO	**[TBD]**	**[TBD]**	**[TBD]**
A2C	**[TBD]**	**[TBD]**	**[TBD]**

TABLE VI
REDUCTION IN AVERAGE TOTAL WAITING TIME RELATIVE TO FIXED-TIME CONTROL (%), COLOGNE1, BY TEST DEMAND LEVEL

Controller	$0.7\times$ (unseen)	$1.0\times$	$1.3\times$ (unseen)	Real hour
Actuated (tuned)	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**
DQN	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**
Double DQN	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**
PPO	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**
A2C	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**

B. Generalization to Unseen Demand

Table VI reports the improvement over fixed-time control at each test demand level. A controller that generalizes keeps its advantage at the unseen levels $0.7\times$ and $1.3\times$, not only at the $1.0\times$ level seen in training. [TODO: Fill from test_summary.csv (vs_fixed_pct). Discuss whether the advantage shrinks at 1.3x, where congestion is highest, and whether completion rates drop.]

C. Learning Behaviour

Fig. 2 shows the validation waiting time during training. [TODO: Replace the placeholder with the learning-curve plot from colab/01_train.ipynb (one panel per scenario, mean $\pm$ s.d. over seeds, fixed-time as a reference line). Discuss: how many episodes each algorithm needs to beat fixed-time; stability across seeds; training time per run.]

D. Ablation Study

Table VII removes one framework component at a time from the best learner, holding everything else fixed. [TODO: Interpret: the effect of demand-randomized training on unseen demand (C1), and shared vs. independent policies on cologne3 (C2).]

E. Limitations

First, the demand variants rescale and re-time the real trips but keep their origins and destinations. The test therefore measures robustness to the amount and timing of traffic, not to new route patterns. Second, results are obtained in simulation, and the sim-to-real gap [20] is not addressed. Third, the observation assumes lane-level density and queue sensing at every approach. Finally, the computational cost of running the policies on roadside hardware is not evaluated here.

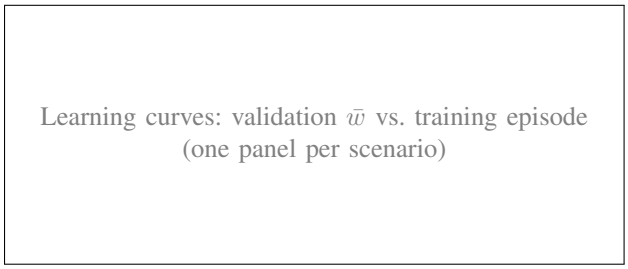


Fig. 2. Validation waiting time during training (mean ± s.d. over five seeds). The dashed line is fixed-time control.

TABLE VII
ABLATION OF THE BEST LEARNER: AVERAGE TOTAL WAITING TIME (S), MEAN ± S.D. OVER 5 SEEDS

Variant	Scenario	Unseen demand	Real hour
Full framework (C1 + C2)	cologne1	[TBD]	[TBD]
−C1: train on the real hour only	cologne1	[TBD]	[TBD]
Full framework (C1 + C2)	cologne3	[TBD]	[TBD]
−C1: train on the real hour only	cologne3	[TBD]	[TBD]
−C2: independent policy per signal	cologne3	[TBD]	[TBD]

VII. Conclusion and Future Work

This paper presented a generalization-focused framework for deep RL traffic signal control, combining demand-randomized training with validation-based model selection and a policy shared across heterogeneous intersections. On three real-city RESCO scenarios, with test demand never seen in training, we compared DQN, Double DQN, PPO and A2C against fixed-time and tuned actuated control. [TODO: Headline findings: best algorithm; improvement on unseen demand; the more important framework component in the ablation.]

This study is the first stage of a resource-aware edge–cloud framework for intelligent transportation. Next, the best learner will be compressed and evaluated under the memory and latency limits of roadside devices, building on compact-controller work such as TinyLight [24] and π-Light [19]. Later stages will add a multi-tier (edge and cloud) anomaly-detection pipeline trained on incidents injected into the same scenarios, and a confidence-aware offloading policy that decides at run time whether inference runs on the roadside device or in the cloud. These components will be integrated into a unified framework and evaluated end to end. We also plan to test under shifts in origin–destination patterns.

Acknowledgment

[TODO: acknowledgments, if any]

References

[1] F. V. Webster, "Traffic signal settings," Road Research Laboratory, HMSO, London, Road Research Technical Paper 39, 1958.
[2] H. Wei, G. Zheng, V. Gayah, and Z. Li, "A survey on traffic signal control methods," arXiv:1904.08117, 2019.
[3] H. Wei, G. Zheng, H. Yao, and Z. Li, "IntelliLight: A reinforcement learning approach for intelligent traffic light control," in *Proc. 24th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining*, 2018, pp. 2496–2505.
[4] H. Wei, C. Chen, G. Zheng, K. Wu, V. Gayah, K. Xu, and Z. Li, "PressLight: Learning max pressure control to coordinate traffic signals in arterial network," in *Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining*, 2019.
[5] C. Chen, H. Wei, N. Xu, G. Zheng, M. Yang, Y. Xiong, K. Xu, and Z. Li, "Toward a thousand lights: Decentralized deep reinforcement learning for large-scale traffic signal control," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 34, no. 4, 2020, pp. 3414–3421.
[6] T. Chu, J. Wang, L. Codecà, and Z. Li, "Multi-agent deep reinforcement learning for large-scale traffic signal control," *IEEE Transactions on Intelligent Transportation Systems*, vol. 21, no. 3, pp. 1086–1095, 2020.
[7] J. Ault and G. Sharon, "Reinforcement learning benchmarks for traffic signal control," in *Proc. Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.
[8] H. Zhang, C. Liu, W. Zhang, G. Zheng, and Y. Yu, "GeneraLight: Improving environment generalization of traffic signal control via meta reinforcement learning," in *Proc. 29th ACM Int. Conf. Information and Knowledge Management (CIKM)*, 2020.
[9] D. Kwesiga, A. Guin, K. Abdelghany, and M. Hunter, "Evaluating the robustness of reinforcement learning based adaptive traffic signal control," arXiv:2603.15283, 2026.
[10] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," in *Proc. AAAI Conf. Artificial Intelligence*, 2018.
[11] P. Varaiya, "Max pressure control of a network of signalized intersections," *Transportation Research Part C: Emerging Technologies*, vol. 36, pp. 177–195, 2013.
[12] P. A. Lopez, M. Behrisch, L. Bieker-Walz, J. Erdmann, Y.-P. Flötteröd, R. Hilbrich, L. Lücken, J. Rummel, P. Wagner, and E. Wießner, "Microscopic traffic simulation using SUMO," in *Proc. 21st IEEE Int. Conf. Intelligent Transportation Systems (ITSC)*, 2018, pp. 2575–2582.
[13] S. Uppoor, O. Trullols-Cruces, M. Fiore, and J. M. Barcelo-Ordinas, "Generation and analysis of a large-scale urban vehicular mobility dataset," *IEEE Transactions on Mobile Computing*, vol. 13, no. 5, pp. 1061–1075, 2014.
[14] S. C. Lobo, S. Neumeier, E. M. G. Fernandez, and C. Facchi, "InTAS – the Ingolstadt traffic scenario for SUMO," in *Proc. SUMO User Conference*, 2020.
[15] V. Mnih, K. Kavukcuoglu, D. Silver *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
[16] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI Conf. Artificial Intelligence*, 2016, pp. 2094–2100.
[17] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.
[18] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," in *Proc. 33rd Int. Conf. Machine Learning (ICML)*, 2016, pp. 1928–1937.
[19] Y. Gu, K. Zhang, Q. Liu, W. Gao, L. Li, and J. Zhou, "π-Light: Programmatic interpretable reinforcement learning for resource-limited traffic signal control," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 38, no. 19, 2024, pp. 21 107–21 115.
[20] F. Al Rafi, S. Mukherjee, L. L. Dekate, J. Y. Lavoe, H. Yao, S. Mohanty, L. Da, X. Zhou, and H. Wei, "Sim2Signal: Sim-to-real benchmarks for traffic signal control," arXiv:2609.01676, 2026.
[21] L. N. Alegre, "SUMO-RL," https://github.com/LucasAlegre/sumo-rl, 2019.
[22] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.
[23] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare, "Deep reinforcement learning at the edge of the statistical precipice," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
[24] D. Xing, Q. Zheng, Q. Liu, and G. Pan, "TinyLight: Adaptive traffic signal control on devices with extremely limited resources," in *Proc. 31st Int. Joint Conf. Artificial Intelligence (IJCAI)*, 2022.